Note that you can define your own project ideas.
If you plan to propose your own idea: Get it approved by me over email. Send an email describing the: (1) Problem statement (2) Objectives (3) Proposed Outcomes
Pick system-level analysis for existing systems with varying configurations. For example, you have a sequence of containers on the communication path for a client request. You vary the container locations across different nodes, use different inter-container communication mechanisms, and observe the performance/behavior. In short, the project must either build new systems or profile existing systems and provide insights.

A. Kubernetes + OS / Networking (8 topics)

---

1. Understanding Tail Latency in Kubernetes Microservices

Tech: Kubernetes + eBPF
Statement:
Kubernetes abstracts resource management, but tail latency in microservices often remains unpredictable. This project aims to instrument Kubernetes pods using eBPF to identify kernel-level causes of p99 latency inflation (e.g., scheduling delays, socket buffering, TCP retransmissions). Students will correlate pod-level SLO violations with OS- and network-level events and propose mitigation strategies.

---

2. Network-Aware Pod Placement in Kubernetes

Tech: Kubernetes
Statement:
Kubernetes schedulers are largely compute-centric and often ignore network topology. This project explores how pod placement affects inter-pod communication latency and bandwidth. Students will design a network-aware scheduling policy (simulation or prototype) and demonstrate performance differences using real workloads.

---

3. Investigating TCP Behavior Inside Kubernetes Pods

Tech: Kubernetes + eBPF
Statement:
TCP performance inside containers may differ from bare-metal due to namespace isolation and cgroup constraints. This project studies TCP congestion control, buffer behavior, and retransmissions inside Kubernetes pods using eBPF instrumentation and controlled experiments.

---

4. Impact of CPU Throttling on Network Performance in Containers

Tech: Kubernetes + eBPF
Statement:
CPU limits enforced via cgroups can cause unexpected network performance degradation. This project investigates how CPU throttling affects packet processing, socket buffers, and tail latency in containerized applications, and identifies kernel paths responsible for performance collapse.

---

5. Diagnosing Kubernetes Networking Failures Using Kernel Traces

Tech: Kubernetes + eBPF
Statement:
Network failures in Kubernetes are often opaque to application developers. This project uses eBPF probes to trace packet drops, retransmissions, and queue overflows in Kubernetes networking paths, enabling fine-grained diagnosis of common failure scenarios.

---

6. Performance Comparison of Kubernetes CNI Plugins

Tech: Kubernetes
Statement:
Different CNI plugins (Flannel, Calico, Cilium) implement networking in fundamentally different ways. This project compares their performance, latency, and CPU overhead under realistic microservice communication patterns and explains observed behavior via architectural differences.

---

7. Backpressure Propagation Across Microservices in Kubernetes

Tech: Kubernetes + eBPF
Statement:
Backpressure in one microservice can cascade across the system. This project traces how kernel-level backpressure (socket buffers, TCP windows) propagates through a chain of Kubernetes services and impacts end-to-end latency and throughput.

---

8. Observability Gaps Between Kubernetes Metrics and Kernel Reality

Tech: Kubernetes + eBPF
Statement:
Kubernetes exposes high-level metrics, but hides kernel-level behavior. This project identifies mismatches between Kubernetes-reported metrics and actual OS/network events, highlighting blind spots in container observability.

B. eBPF-Centric Systems Projects (6 topics)

---

9. Reconstructing TCP State Machine Using eBPF

Tech: eBPF
Statement:
TCP state transitions are rarely visible to applications. This project uses eBPF probes to reconstruct TCP connection state machines in real time, allowing students to observe retransmissions, congestion events, and abnormal teardown scenarios.

---

10. Measuring Copy vs Zero-Copy Behavior in the Kernel

Tech: eBPF
Statement:
Modern kernels implement multiple data movement paths. This project instruments kernel functions related to copying (`copy_from_user`, `skb_copy_datagram_iter`) to quantify copy overheads under different socket APIs and message sizes.

---

11. eBPF-Based Detection of Network Backpressure

Tech: eBPF
Statement:
Backpressure is a fundamental mechanism in OS and networks but lacks direct observability. This project builds an eBPF-based detector that identifies backpressure signals in socket buffers, TCP windows, and NIC queues, and correlates them with application slowdowns.

---

12. Profiling Kernel Scheduling Delays for Network-Heavy Workloads

Tech: eBPF
Statement:
Network packet processing competes with application threads for CPU. This project uses eBPF to measure scheduling delays in kernel and user contexts and studies their impact on network-intensive applications.

---

13. Identifying Kernel Hotspots in Microservice Communication

Tech: eBPF
Statement:
Microservice architectures incur significant kernel overhead. This project identifies kernel hotspots involved in inter-process and inter-container communication and evaluates optimization opportunities.

---

14. eBPF-Based Socket Lifetime and Resource Leak Detection

Tech: eBPF
Statement:
Socket leaks and delayed cleanup can cause subtle performance degradation. This project tracks socket lifecycle events using eBPF and detects anomalous resource usage patterns.

C. LLM Simulator + Systems (6 topics)

---

15. Simulating Distributed LLM Training Over Different Network Topologies

Tech: LLM simulator
Statement:

Large-scale LLM training is sensitive to network topology and bandwidth. This project uses an LLM simulator to study how different datacenter network designs affect training time and communication efficiency.

---

16. Effect of Collective Communication Algorithms on LLM Training

Tech: LLM simulator
Statement:
All-reduce and other collective operations dominate LLM training. This project compares different collective algorithms under varying network conditions using simulation and explains their performance trade-offs.

---

17. Sensitivity of LLM Training to Network Jitter and Loss

Tech: LLM simulator
Statement:
LLM training assumes reliable high-performance networks. This project injects controlled delay, jitter, and loss into simulated training and evaluates training stability and efficiency.

---

18. Kubernetes-Aware LLM Training Simulation

Tech: Kubernetes + LLM simulator
Statement:
Production LLM training often runs on Kubernetes. This project simulates LLM workloads under Kubernetes scheduling, resource limits, and pod placement policies to analyze performance variability.

---

19. Modeling Straggler Effects in Distributed LLM Training

Tech: LLM simulator
Statement:
Stragglers significantly impact synchronous training. This project models straggler behavior arising from OS scheduling, network congestion, or resource contention and evaluates mitigation techniques.

---

20. Evaluating Transport Protocol Choices for LLM Training

Tech: LLM simulator + Networking

Statement:

LLM training uses different transport mechanisms (TCP, RDMA, custom transports). This project simulates and compares their impact on training throughput and scalability.